

KULIAH PRAKTEK KOMPUTASI AWAN

MODUL 4.2

Deploy Aplikasi Kas RW ke AWS EC2

Backend, Database, dan Frontend di Cloud

Oleh : Dr. Andrew B. Osmond

Mata Kuliah	Praktikum Cloud Computing
Topik	Deployment Aplikasi ke Cloud (AWS EC2)
Studi Kasus	Aplikasi Pencatatan Kas RW
Perangkat	AWS Academy Learner Lab, EC2, Git, Python, Nginx
Estimasi Waktu	90 - 120 menit

Pada modul ini, kode backend FastAPI yang telah dibuat di Modul 4.1 akan di-deploy ke EC2 instance di AWS. Database MySQL dikonfigurasi di ec2-database, backend FastAPI berjalan di ec2-backend, dan frontend HTML statis di-serve oleh Nginx di ec2-frontend. Ketiganya saling terhubung membentuk arsitektur tiga tier yang berjalan di cloud.

A. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- * Meluncurkan EC2 instances dan mengonfigurasi Security Group di AWS
- * Melakukan SSH ke EC2 instance menggunakan file labsuser.pem
- * Menginstal dan mengonfigurasi MySQL Server di EC2
- * Men-deploy backend FastAPI ke EC2 menggunakan git clone
- * Men-deploy frontend HTML statis menggunakan Nginx di EC2
- * Memverifikasi koneksi end-to-end dari browser ke frontend, backend, hingga database

B. Latar Belakang

Pada Modul 4.1, seluruh kode backend telah dibuat dan disimpan di GitHub. Kini saatnya kode tersebut benar-benar berjalan di cloud -- dapat diakses dari mana saja, bukan hanya dari komputer lokal.

Arsitektur yang dibangun pada modul ini mereplikasi apa yang telah dilakukan di Modul 3, namun kali ini disertai aplikasi nyata yang berjalan di atasnya:

Arsitektur Tiga Tier

Browser -> Nginx (ec2-frontend:80) -> FastAPI (ec2-backend:8000) -> MySQL (ec2-database)

ec2-frontend : menyajikan antarmuka HTML ke browser pengguna

ec2-backend : menjalankan logika bisnis dan API
ec2-database : menyimpan data transaksi kas

Catatan: IP Dinamis di AWS Academy Learner Lab

IP EC2 berubah setiap kali instance di-start ulang atau sesi lab baru dibuka. Oleh karena itu, IP di file .env dan di index.html frontend perlu diperbarui secara manual di awal setiap sesi. Di lingkungan AWS nyata, Elastic IP dapat digunakan untuk IP statis.

C. Prasyarat

- * Modul 4.1 sudah diselesaikan: repo kas-rw-backend sudah ada di GitHub
- * Akses ke AWS Academy Learner Lab sudah tersedia
- * File labsuser.pem sudah diunduh dari halaman Learner Lab
- * Security group sg-kas-rw sudah dikonfigurasi dengan inbound rules berikut:

Type	Port	Source	Tujuan
SSH	22	0.0.0.0/0	Akses SSH dari lokal
HTTP	80	0.0.0.0/0	Akses frontend dari browser
Custom TCP	8000	0.0.0.0/0	Akses API backend
All traffic	All	sg-kas-rw	Komunikasi antar instance

D. Meluncurkan EC2 Instances

Luncurkan tiga instance dengan konfigurasi berikut. Ulangi langkah ini sebanyak tiga kali, hanya beda di Name:

1. Di EC2 Dashboard, klik Launch instance
2. Name: ec2-database (ulangi untuk ec2-backend dan ec2-frontend)
3. AMI: Ubuntu Server 24.04 LTS, 64-bit (x86)
4. Instance type: t2.micro
5. Key pair: vockey
6. Network settings -> Edit: Auto-assign public IP: Enable, Firewall: pilih sg-kas-rw
7. Klik Launch instance

Setelah ketiga instance berstatus Running, catat IP address masing-masing:

Instance	Public IP	Private IP
ec2-database	...	...
ec2-backend	...	...
ec2-frontend	...	...

Public IP vs Private IP

Public IP : digunakan untuk SSH dari komputer lokal dan akses browser.

Private IP : digunakan dalam file .env dan konfigurasi antar instance.

Gunakan selalu Private IP untuk komunikasi antar EC2 -- lebih stabil dan tidak dikenakan biaya transfer data.

E. Setup Database di ec2-database

E.1. SSH ke ec2-database

macOS / Linux -- Terminal

```
chmod 400 ~/Downloads/labsuser.pem
ssh -i ~/Downloads/labsuser.pem ubuntu@<PUBLIC_IP_EC2_DATABASE>
```

Windows 11 -- PowerShell

```
ssh -i $env:USERPROFILE\Downloads\labsuser.pem ubuntu@<PUBLIC_IP_EC2_DATABASE>
```

E.2. Install MySQL

```
sudo apt update
sudo apt install -y mysql-server
```

E.3. Buat Database, User, dan Tabel

```
sudo mysql
```

Jalankan query berikut di dalam konsol MySQL:

```
CREATE DATABASE kasrw;
CREATE USER 'kasrw_user'@'%' IDENTIFIED BY 'password123';
GRANT ALL PRIVILEGES ON kasrw.* TO 'kasrw_user'@'%';
FLUSH PRIVILEGES;
USE kasrw;

CREATE TABLE transaksi (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    tanggal       DATE NOT NULL,
    keterangan    VARCHAR(255) NOT NULL,
    jenis         ENUM('pemasukan', 'pengeluaran') NOT NULL,
    jumlah        DECIMAL(15, 2) NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

EXIT;
```

Catatan: Host '%' pada CREATE USER

Di Modul 1, user MySQL dibatasi hanya untuk IP vm-backend secara eksplisit. Di EC2, kita menggunakan '%' (semua IP) karena IP instance bersifat dinamis. Keamanan jaringan ditangani oleh Security Group AWS, bukan oleh MySQL.

E.4. Izinkan Koneksi dari Luar Localhost

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```

Cari baris bind-address = 127.0.0.1 dan ubah menjadi:

```
bind-address = 0.0.0.0
```

Simpan dengan Ctrl+X -> Y -> Enter, lalu restart MySQL:

```
sudo systemctl restart mysql
```

Keluar dari ec2-database:

```
exit
```

F. Deploy Backend di ec2-backend

F.1. SSH ke ec2-backend

Buka terminal baru, lalu SSH ke ec2-backend:

macOS / Linux -- Terminal

```
ssh -i ~/Downloads/labsuser.pem ubuntu@<PUBLIC_IP_EC2_BACKEND>
```

Windows 11 -- PowerShell

```
ssh -i $env:USERPROFILE\Downloads\labsuser.pem ubuntu@<PUBLIC_IP_EC2_BACKEND>
```

F.2. Install Git dan Clone Repo

```
sudo apt update
sudo apt install -y git
git clone https://github.com/<username>/kas-rw-backend.git
cd kas-rw-backend
```

F.3. Buat File .env

```
cp .env.example .env  
nano .env
```

Isi dengan Private IP ec2-database:

```
DB_HOST=<PRIVATE_IP_EC2_DATABASE>  
DB_PORT=3306  
DB_NAME=kasrw  
DB_USER=kasrw_user  
DB_PASSWORD=password123
```

F.4. Install Dependencies

```
pip3 install -r requirements.txt --break-system-packages
```

Mengapa --break-system-packages?

Ubuntu 24.04 memproteksi environment Python sistem dari instalasi package eksternal. Flag --break-system-packages mengizinkan instalasi langsung ke sistem. Ini aman untuk server EC2 yang didedikasikan untuk satu aplikasi. Alternatif yang lebih bersih: gunakan virtual environment (python3 -m venv).

F.5. Jalankan Backend

```
python3 -m uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Verifikasi di browser:

```
http://<PUBLIC_IP_EC2_BACKEND>:8000/docs
```

Swagger UI harus muncul. Lakukan test POST /transaksi untuk memastikan koneksi ke database berhasil.

G. Deploy Frontend di ec2-frontend

G.1. Buat Repo kas-rw-frontend

Di GitHub, buat repositori baru bernama kas-rw-frontend (kosong, tanpa README). Di komputer lokal:

macOS / Linux -- Terminal

```
cd ~
```

```
git clone https://github.com/<username>/kas-rw-frontend.git
cd kas-rw-frontend
```

Windows 11 -- PowerShell

```
cd $env:USERPROFILE
git clone https://github.com/<username>/kas-rw-frontend.git
cd kas-rw-frontend
```

G.2. Buat index.html

Buat file index.html dengan isi berikut. Ganti <PUBLIC_IP_EC2_BACKEND> dengan IP backend:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Kas RW</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 800px; margin: 40px auto; padding:
0 20px; }
    h1 { color: #1a73e8; }
    table { width: 100%; border-collapse: collapse; margin-top: 20px; }
    th, td { border: 1px solid #ddd; padding: 8px 12px; text-align: left; }
    th { background: #1a73e8; color: white; }
    tr:nth-child(even) { background: #f5f5f5; }
    .pemasukan { color: green; font-weight: bold; }
    .pengeluaran { color: red; font-weight: bold; }
    form { margin-top: 30px; display: grid; gap: 10px; max-width: 400px; }
    input, select { padding: 8px; border: 1px solid #ddd; border-radius: 4px; }
    button { padding: 10px; background: #1a73e8; color: white; border: none;
      border-radius: 4px; cursor: pointer; }
  </style>
</head>
<body>
  <h1>Aplikasi Pencatatan Kas RW</h1>
  <h2>Tambah Transaksi</h2>
  <form id="form-transaksi">
    <input type="date" id="tanggal" required>
    <input type="text" id="keterangan" placeholder="Keterangan" required>
    <select id="jenis">
      <option value="pemasukan">Pemasukan</option>
      <option value="pengeluaran">Pengeluaran</option>
    </select>
    <input type="number" id="jumlah" placeholder="Jumlah (Rp)" required>
    <button type="submit">Simpan</button>
  </form>
  <h2>Daftar Transaksi</h2>
  <table>

  <thead><tr><th>Tanggal</th><th>Keterangan</th><th>Jenis</th><th>Jumlah</th></tr></thead>
  <tbody id="tabel-transaksi"></tbody>
</table>
  <script>
    const API = "http://<PUBLIC_IP_EC2_BACKEND>:8000";
    function formatRupiah(n){ return "Rp " + Number(n).toLocaleString("id-ID"); }
```

```
async function loadTransaksi() {
  const res = await fetch(API + "/transaksi");
  const data = await res.json();
  const tbody = document.getElementById("tabel-transaksi");
  tbody.innerHTML = "";
  data.forEach(function(t) {
    tbody.innerHTML += `<tr><td>${t.tanggal}</td><td>${t.keterangan}</td>
      <td>
class="${t.jenis}">${t.jenis}</td><td>${formatRupiah(t.jumlah)}</td></tr>`;
    });
  }
  document.getElementById("form-transaksi").addEventListener("submit", async
function(e) {
  e.preventDefault();
  const body = { tanggal: document.getElementById("tanggal").value,
    keterangan: document.getElementById("keterangan").value,
    jenis: document.getElementById("jenis").value,
    jumlah: parseFloat(document.getElementById("jumlah").value) };
  await fetch(API + "/transaksi", { method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(body) });
  e.target.reset();
  loadTransaksi();
});
loadTransaksi();
</script>
</body>
</html>
```

G.3. Push ke GitHub

```
git add index.html
git commit -m "feat: initial frontend Kas RW"
git push -u origin main
```

G.4. Deploy ke ec2-frontend

Buka terminal baru, SSH ke ec2-frontend:

macOS / Linux -- Terminal

```
ssh -i ~/Downloads/labsuser.pem ubuntu@<PUBLIC_IP_EC2_FRONTEND>
```

Windows 11 -- PowerShell

```
ssh -i $env:USERPROFILE\Downloads\labsuser.pem ubuntu@<PUBLIC_IP_EC2_FRONTEND>
```

Install Nginx dan Git, lalu clone repo frontend:

```
sudo apt update
sudo apt install -y nginx git
git clone https://github.com/<username>/kas-rw-frontend.git
```

```
sudo cp ~/kas-rw-frontend/index.html /var/www/html/index.html
```

Verifikasi Nginx berjalan:

```
sudo systemctl status nginx
```

H. Verifikasi End-to-End

Buka browser dan akses:

```
http://<PUBLIC_IP_EC2_FRONTEND>
```

Halaman Kas RW harus muncul dan menampilkan data transaksi. Coba tambah transaksi baru melalui form -- data harus langsung muncul di tabel tanpa reload halaman.

Goal Modul 4.2 Tercapai!

Jika halaman frontend tampil dan form transaksi berfungsi, maka:

Browser -> Nginx (ec2-frontend) -> FastAPI (ec2-backend) -> MySQL (ec2-database)

Seluruh arsitektur tiga tier berjalan penuh di AWS EC2.

I. Troubleshooting

ERR_CONNECTION_RESET saat akses port 8000

Pastikan Security Group sg-kas-rw sudah memiliki rule Custom TCP port 8000 dengan source 0.0.0.0/0. Setelah menambahkan rule, reload uvicorn di ec2-backend.

500 Internal Server Error saat POST /transaksi

- * Pastikan uvicorn berjalan di ec2-backend sebelum membuka frontend
- * Periksa isi .env -- DB_HOST harus menggunakan Private IP ec2-database, bukan Public IP
- * Pastikan MySQL sudah berjalan di ec2-database: `sudo systemctl status mysql`

Frontend tidak menampilkan data (tabel kosong)

Kemungkinan IP backend di index.html sudah berubah (sesi lab baru). Update Public IP ec2-backend di index.html, push ke GitHub, lalu pull ulang di ec2-frontend dan copy file lagi ke /var/www/html/.

IP berubah di sesi lab baru

Di awal setiap sesi lab, lakukan langkah berikut:

- * Catat ulang Public IP dan Private IP dari AWS Console
- * Update .env di ec2-backend dengan Private IP ec2-database terbaru
- * Update const API di index.html dengan Public IP ec2-backend terbaru, push, dan pull ulang di ec2-frontend

J. Kesimpulan

Pada modul ini, aplikasi Kas RW berhasil di-deploy ke AWS EC2 dengan arsitektur tiga tier yang berjalan penuh di cloud. Kode backend yang sebelumnya hanya ada di komputer lokal kini dapat diakses dari mana saja melalui browser.

Yang Sudah Dikerjakan	Yang Akan Datang
MySQL terkonfigurasi di ec2-database Backend FastAPI berjalan di ec2-backend Frontend HTML di-serve Nginx di ec2-frontend Arsitektur tiga tier berjalan penuh di AWS	Membuat Dockerfile untuk backend Build dan run container Docker di EC2 Push image ke Docker Hub